

Standalone Boot Mode

This guide describes how firmware is stored on the board and started at power-on. The application resides in the QSPI flash. A small bootloader (AMD/Xilinx's standard `srec_spi_bootloader`, part of the hardware image) copies it into SRAM and starts it, with no PC attached, in under a second after power is applied.

1. How Standalone Boot Works

Power-on → FPGA configures itself from QSPI flash (<1 s)

→ Bootloader (in Block RAM, part of the hardware image) starts

→ Reads the application image (SREC) from flash @ `0x220000`

→ Copies it to the addresses the image names (SRAM)

→ Jumps to the application's entry point

The application is stored as an SREC image (Motorola S-record), a text format in which every line carries its own destination address. The bootloader copies each record to the address it names, so the linker script decides where the program runs. The course template links to SRAM at `0x6000_0000`, and that is where the image ends up.

Memory roles:

Memory	Size	Role
QSPI flash	4 MB	hardware image + bootloader (first 2.2 MB) · application slot @ <code>0x220000</code> (non-volatile)
SRAM @ <code>0x60000000</code>	512 KB	where your application executes (code/data/heap, I/D-cached)
BRAM @ <code>0x00000000</code>	128 KB	32 KB bootloader (untouchable) · 32 KB ITCM @ <code>0x8000</code> (your fast code) · 64 KB DTCM @ <code>0x10000</code> (stack + fast data)

Note: The bootloader is part of the hardware image and is restored from flash at every power-on. Like a mask ROM, it cannot be corrupted from software.

2. One-Time Board Setup (Instructor)

Program the deployment image `release/official_boot.mcs` (hardware image + bootloader + preloaded demo application) into the QSPI flash:

- Open Vivado **Hardware Manager > Open Target > Auto Connect**.
- Right-click `xc7a35t_0` > **Add Configuration Memory Device** — pick the part matching the IC3 chip: `mx25l3273f-spi-x1_x2_x4` (Vivado's catalog entry covering the board's Macronix MX25L3233F — verified working) or `n25q32-3.3v-spi-x1_x2_x4` (Micron, older boards).

3. Right-click the memory device > **Program Configuration Memory Device**, select `release/official_boot.mcs`, keep **Erase / Program / Verify** checked, **OK**.
4. Power-cycle. The preloaded demo application starts (its banner appears on the USB serial console at 115200 baud).

Students never repeat this step; from here on the board is a pure MCU.

3. Build Your Application

Create a Vitis application as in the [JTAG guide](#) (sections 1–7), using the course template `workspace-example/app_template/src/` — its `main.c` and `lscript.ld` replace the generated Hello World sources.

The template's linker script places:

- `.text` / `.rodata` / `.data` / `.bss` / heap → **SRAM** (512 KB, cached)
- stack → top of **DTCM** (64 KB BRAM @ `0x10000`; 1-cycle, never collides with the bootloader)
- functions tagged `ITCM_FUNC` → **ITCM** (32 KB BRAM @ `0x8000`); variables tagged `DTCM_DATA` → DTCM

ITCM code ships inside the SRAM image and is copied out by `mcu_init()` at the top of `main()`, the same startup-copy idiom an STM32H7 uses for its ITCM. This is why the template has one rule: `mcu_init();` stays the first line of `main()`. (It also sets the UART to 115200.) Put interrupt handlers and timing-critical loops in ITCM — TCM never misses, so worst-case latency equals best-case.

The template prints a memory tour at boot, which shows the layout directly:

```
RISC-V MCU on Cmod A7 - memory tour
main()      @ 0x600009D0  (SRAM,  cached)
blink_step() @ 0x00008000  (ITCM,  1-cycle)
blink_count @ 0x00010000  (DTCM,  1-cycle)
stack       @ 0x0001FFC0  (DTCM,  grows down)
```

Note: The same ELF works unchanged in both modes: Vitis Run/Debug over JTAG during development, flash deployment to ship it. No rebuild is needed.

4. Deploy to Flash

Deployment = converting the built ELF to SREC and writing it into the flash application slot at `0x220000`:

1. **Convert to SREC** with the toolchain's `objcopy` (`riscv32-amd-linux-gnu-objcopy -O srec app.elf app.srec`). The record addresses come from the linker script — with the course template they all fall in SRAM.
2. **Write the SREC file's bytes to flash offset** `0x220000` over JTAG. The erase and write are scoped to the application slot, so the hardware image at the start of flash is never touched.
3. **Power-cycle (or reboot from flash)** — the bootloader picks up the new image.

(Step-by-step GUI walkthrough with screenshots: to be added.)

The slot runs from the end of the bitstream region to the end of the flash: 1.875 MB, more SREC text than a full-SRAM application converts to (SREC is about 3× the binary size). In practice the size limit is the 512 KB SRAM. Only code and initialized data count toward it; zero-initialized data and the heap take no space in the image.

5. Boot and Verify

- **Power-cycle** → the board boots your flashed app automatically. No PC needed.
- `xil_printf` (the SDK's lightweight `printf`) output arrives on the same USB cable at **115200 baud**. To watch it, open any serial terminal, e.g. `python3 -m serial.tools.miniterm /dev/ttyUSB1 115200` (ships with pyserial; quit with Ctrl-J). The board shows up as two ports — the UART is usually the higher-numbered one.
- A useful convention: light an LED near the top of `main()` so a running board is recognizable at a glance. The template's color-cycling RGB LED serves this purpose.

Note: The reset button (BTN0) restarts the CPU into the bootloader (BRAM is intact), which re-copies your app from flash, so in this design a reset behaves like a power-cycle. Modified `.data` globals are restored because the image is re-read from flash each boot.

6. Working with JTAG Debug Mode

Both modes coexist on the same board with no switching:

	JTAG (Vitis Run/Debug)	Standalone (flash)
Code goes to	RAM directly (volatile)	Flash (persistent)
After power-cycle	back to the flashed app	your app boots
Breakpoints / stepping	yes	no
Typical use	daily development loop	shipping a finished program

The everyday loop is edit, build, JTAG Run/Debug. When the program works, deploy it to flash once. JTAG always takes priority over the bootloader: it halts the CPU before the bootloader runs, so the flash contents cannot interfere with a debug session.

7. How the Deployment Image Is Made (Instructor Reference)

`release/official_boot.mcs` = the hardware image with the bootloader merged into its BRAM initialization, plus the demo application's SREC at `0x220000`. The full mechanics (BRAM init frames, `updatemem`, `write_cfgmem`, flash layout) are in the [Boot Image Pipeline guide](#).

The bootloader is not custom code. It is Vitis's stock "SREC SPI Bootloader" application template, built with one configuration change in `src/blconfig.h`:

```
#define FLASH_IMAGE_BASEADDR 0x00220000
```

Board-verified facts for anyone rebuilding it:

1. **The flash controller is auto-selected correctly.** Vitis 2025.2 (System Device Tree flow) picks `XPAR_AXI_QUAD_SPI_0_BASEADDR`, which is this board's flash controller. No manual fix is needed.
 2. **Keep `VERBOSE` off.** The template's debug prints assume an initialized UART divisor; on this board they hang before the jump (the CPU parks in `XUartNs550_SendByte`). It is undefined by default; leave it that way, or call `XUartNs550_SetBaud(...)` first.
 3. The bootloader links to BRAM (`0x0`), so it can copy the application into SRAM without overwriting itself. Merging its ELF into the bitstream uses `updatemem` (about 6 seconds, no re-implementation); see the Boot Image Pipeline guide.
 4. Applications must set their own UART baud before printing. The course template's `mcu_init()` does this.
-

8. Troubleshooting

1. **Garbage on the terminal** — Baud must be 115200; the app sets it via `mcu_init()`. The stock Hello World template does not set it; add `XUartNs550_SetBaud(XPAR_UART_USB_BASEADDR, 100000000, 115200);` or use the course template.
2. **Deployed, but silent after power-on** — The ELF was probably not linked with the course template's `lscript.ld`: the bootloader honors the SREC record addresses, and a default all-BRAM layout collides with the bootloader itself. Rebuild with the template and redeploy.
3. **No memory tour / garbage addresses** — `mcu_init()` is not the first line of `main()`, or it was removed: nothing tagged `ITCM_FUNC` / `DTCM_DATA` works before it runs.
4. **Board boots an old app** — the flash write did not complete (check the programming step's verify result), or you power-cycled before it finished. Redeploy.
5. **Nothing at all (no LED, no output)** — the FPGA did not configure, which means the flash image is invalid; redo §2. JTAG still works for recovery.
6. **Want a factory-blank board** — Hardware Manager > right-click the memory device > **Erase**. The FPGA then waits for JTAG at power-on.